

Hier is een overzicht in canvas-stijl van alles wat nodig was om de BTC minute-data correct om te zetten naar daily data, inclusief Git-aanpassingen en scripts.

BTC Minute -> Daily Pipeline (Project Plan)

1. Vereisten / Installaties

- Python 3.12 (venv aanbevolen)
- pip packages:
- pandas
- numpy
- ccxt
- sqlite3 (builtin)

```
python3 -m venv .venv
source .venv/bin/activate
pip install pandas numpy ccxt
```

- Projectfolders:

```
mkdir -p trading_ai_project/data/raw_csv
mkdir -p trading_ai_project/database
```

2. Scripts

2.1 fetch_binance_1m.py

Haal BTC/USDT 1-minuut data van Binance op en sla CSV op.

```
#!/usr/bin/env python3
import ccxt, time, pandas as pd, os
from datetime import datetime, timezone

def fetch_ohlcv_binance(symbol='BTC/USDT', timeframe='1m', since_dt=None,
until_dt=None, limit=1000, out_csv='trading_ai_project/data/raw_csv/
bitcoin_dataset_minute.csv'):
    ex = ccxt.binance({'enableRateLimit': True})
    if since_dt is None:
        since_ms = int((datetime.now(timezone.utc) -
pd.Timedelta(days=7)).timestamp() * 1000)
    else:
        since_ms = int(since_dt.replace(tzinfo=timezone.utc).timestamp() *
1000)
    until_ms = int(until_dt.replace(tzinfo=timezone.utc).timestamp() * 1000)
    if until_dt else None
```

```

all_rows = []
retry = 0
while True:
    try:
        ohlcv = ex.fetch_ohlcv(symbol, timeframe=timeframe,
since=since_ms, limit=limit)
        except Exception as e:
            retry += 1
            if retry > 8: raise
            print(f"API error, retry {retry}: {e}. sleeping 5s")
            time.sleep(5)
            continue

        if not ohlcv: break
        all_rows.extend(ohlcv)
        last_ts = ohlcv[-1][0]
        since_ms = last_ts + 1
        if until_ms and last_ts >= until_ms: break
        if len(ohlcv) < limit: break
        time.sleep(ex.rateLimit / 1000.0)

    df = pd.DataFrame(all_rows,
columns=['ts_ms', 'open', 'high', 'low', 'close', 'volume'])
    df['timestamp'] = pd.to_datetime(df['ts_ms'], unit='ms', utc=True)
    df = df[['timestamp', 'open', 'high', 'low', 'close', 'volume']]
    os.makedirs(os.path.dirname(out_csv), exist_ok=True)
    df.to_csv(out_csv, index=False)
    print(f"Saved {len(df)} rows to {out_csv}")

if __name__ == '__main__':
    fetch_ohlcv_binance()

```

2.2 map_minute_to_expected_schema.py

Map minute data naar 24 koloms schema en schrijf naar SQLite.

```

#!/usr/bin/env python3
import os, pandas as pd, numpy as np, sqlite3

IN_CSV = 'trading_ai_project/data/raw_csv/bitcoin_dataset_minute.csv'
OUT_CSV = 'trading_ai_project/data/raw_csv/bitcoin_dataset_mapped.csv'
OUT_DB = 'trading_ai_project/database/trading_data_minute_mapped.db'
TABLE_NAME = 'btc_minute_mapped'

EXPECTED_COLS = [ ... 24 kolommen zoals eerder ]

def main():
    if not os.path.exists(IN_CSV): raise FileNotFoundError(IN_CSV)
    df = pd.read_csv(IN_CSV, parse_dates=['timestamp'])

```

```

out = pd.DataFrame(index=df.index, columns=EXPECTED_COLS)
out['timestamp'] = df['timestamp']
out['market_price'] = pd.to_numeric(df['close'], errors='coerce') if
'close' in df.columns else np.nan
out['trade_volume'] = pd.to_numeric(df['volume'], errors='coerce') if
'volume' in df.columns else np.nan
out['output_volume'] = out['trade_volume']
out['estimated_transaction_volume'] = out['trade_volume']
out['estimated_transaction_volume_usd'] = out['market_price'] *
out['trade_volume']

for c in EXPECTED_COLS: out[c] = out.get(c, np.nan)
out = out[EXPECTED_COLS]

os.makedirs(os.path.dirname(OUT_CSV), exist_ok=True)
out.to_csv(OUT_CSV, index=False)

os.makedirs(os.path.dirname(OUT_DB), exist_ok=True)
conn = sqlite3.connect(OUT_DB)
out.to_sql(TABLE_NAME, conn, if_exists='replace', index=False)
conn.close()

if __name__ == '__main__': main()

```

2.3 minute -> daily -> main DB

```

import sqlite3, pandas as pd, numpy as np, os

SRC_DB = 'trading_ai_project/database/trading_data_minute_mapped.db'
SRC_TABLE = 'btc_minute_mapped'
DST_DB = 'trading_ai_project/database/trading_data.db'
DAILY_TABLE = 'btc_daily_data'

src_conn = sqlite3.connect(SRC_DB)
dst_conn = sqlite3.connect(DST_DB)

# load minute
df_min = pd.read_sql(f"SELECT * FROM {SRC_TABLE}", src_conn,
parse_dates=['timestamp'])
df_min['timestamp'] = pd.to_datetime(df_min['timestamp'], utc=True)
df_min = df_min.set_index('timestamp')

# resample daily
daily = pd.DataFrame()
daily['market_price'] = df_min['market_price'].resample('D').last()
daily['open'] = df_min['market_price'].resample('D').first()
daily['high'] = df_min['market_price'].resample('D').max()
daily['low'] = df_min['market_price'].resample('D').min()
daily['trade_volume'] = df_min['trade_volume'].resample('D').sum()
daily['output_volume'] = df_min['output_volume'].resample('D').sum()

```

```

daily['estimated_transaction_volume'] =
df_min['estimated_transaction_volume'].resample('D').sum()
daily['estimated_transaction_volume_usd'] =
df_min['estimated_transaction_volume_usd'].resample('D').sum()

EXPECTED_COLS = [ ... 24 kolommen ... ]
df_daily = pd.DataFrame(index=daily.index)
for col in
['market_price', 'trade_volume', 'output_volume', 'estimated_transaction_volume', 'estimated_trans
    df_daily[col] = daily[col]
for c in EXPECTED_COLS: df_daily[c] = df_daily.get(c, np.nan)
df_daily = df_daily.reset_index().rename(columns={'index': 'timestamp'})
df_daily = df_daily[EXPECTED_COLS]

df_daily.to_sql(DAILY_TABLE, dst_conn, if_exists='replace', index=False)
dst_conn.execute(f"CREATE INDEX IF NOT EXISTS idx_{DAILY_TABLE}_ts ON
{DAILY_TABLE}(timestamp);")
dst_conn.commit()
src_conn.close()
dst_conn.close()

```

3. Git Aanpassingen

- Voeg `fetch_binance_1m.py` en `map_minute_to_expected_schema.py` toe.
- Update `.gitignore` voor `.venv/` en eventueel CSV-bestanden als je niet wilt pushen.
- Documenteer in `README.md` dat:
- CSV-map: `trading_ai_project/data/raw_csv`
- Database-map: `trading_ai_project/database`
- Scripts: `fetch_binance_1m.py`, `map_minute_to_expected_schema.py`,
`minute_to_daily.py`
- Vereiste Python packages.

4. Terminal Commands Samengevat

```

# venv
python3 -m venv .venv
source .venv/bin/activate
pip install pandas numpy ccxt

# folders
mkdir -p trading_ai_project/data/raw_csv
mkdir -p trading_ai_project/database

# fetch BTC minute
python3 fetch_binance_1m.py

# map naar expected schema
python3 map_minute_to_expected_schema.py

```

```
# convert naar daily
python3 minute_to_daily.py

# verify
sqlite3 trading_ai_project/database/trading_data.db "SELECT COUNT(*) FROM
btc_daily_data;"
```